WEEK 2 · LESSON 8 OF 24 · HANDOUT

Experiment Tracking and the Model Registry

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=hm3HIZ5mns0 — open the lesson online to watch inside the course.

🎓 **Go deeper:** PyCon US 2024: Trust Fall — Three Hidden Gems in MLflow — https://www.youtube.com/watch?v=cAgrIilgP4A

🎓 **Go deeper:** PyCon Italia 2022: Managing large-scale ML pipelines with MLflow and serverless computing — https://www.youtube.com/watch?v=8AkqF99tkXg

Your goal in this lesson: Record every training run with its settings and results, compare runs fairly, and manage model versions with a registry that supports promotion and rollback.

The spreadsheet problem

After a week of experiments: "Which settings gave AUC 0.89? Was that before or after we fixed the data?" **Experiment tracking** saves, for every run, the config, data fingerprint, metrics and time. A **model registry** keeps the approved model versions and which one is in production.

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

A tiny tracker

```
import hashlib, json, time
from sklearn.ensemble import GradientBoostingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants["repaid"])
fingerprint = hashlib.sha256(pd.util.hash_pandas_object(train,
index=False).values.tobytes()).hexdigest()[:12]
open("runs.jsonl", "w").close()

def build(kind, **params):
    if kind == "logreg":
        return make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000, **params))
    if kind == "forest":
        return RandomForestClassifier(n_estimators=200, random_state=0, **params)
    return GradientBoostingClassifier(random_state=0, **params)

def run(kind, **params):
    start = time.perf_counter()
    m = build(kind, **params).fit(train[FEATURES], train["repaid"])
    p = m.predict_proba(test[FEATURES])[:, 1]
    record = {"kind": kind, "params": params, "data": fingerprint,
              "auc": round(roc_auc_score(test["repaid"], p), 4),
              "accuracy": round(accuracy_score(test["repaid"], p >= 0.5), 4),
              "seconds": round(time.perf_counter() - start, 2)}
    with open("runs.jsonl", "a") as f:
        f.write(json.dumps(record) + "\n")
    return m

for C in [0.01, 1.0, 100.0]:
    run("logreg", C=C)
for depth in [3, 8, None]:
    run("forest", max_depth=depth)
run("boosting", max_depth=2, n_estimators=150)

runs = pd.read_json("runs.jsonl", lines=True)
print(runs.sort_values("auc", ascending=False).to_string(index=False))
```

Now every result can be traced back to its exact settings and data. Notice:

- The **simple logistic regression** is as good as the bigger models here — the data is close to linear.
- Every forest is worse than the logistic regression, and the forest with no depth limit is worse than the one limited to depth 8: it overfits.
- Time matters too: a model 20 times slower for the same AUC is not an improvement.

When results are this close, choose the **simplest** model: easier to explain (Week 3), faster and cheaper.

A model registry

```
import datetime
import joblib

REGISTRY = "registry.json"
json.dump({"models": []}, open(REGISTRY, "w"))

def register(model, version, metrics, notes=""):
    reg = json.load(open(REGISTRY))
    path = f"model_{version}.joblib"
    joblib.dump(model, path)
    reg["models"].append({"version": version, "path": path, "metrics": metrics, "stage":
"staging",
                          "data": fingerprint, "registered": datetime.date.today().isoformat(),
"notes": notes})
    json.dump(reg, open(REGISTRY, "w"), indent=2)

def promote(version):
    reg = json.load(open(REGISTRY))
    for m in reg["models"]:
        if m["stage"] == "production":
            m["stage"] = "archived"
        if m["version"] == version:
            m["stage"] = "production"
    json.dump(reg, open(REGISTRY, "w"), indent=2)

def production():
    return next(m for m in json.load(open(REGISTRY))["models"] if m["stage"] == "production")

def show():
    for m in json.load(open(REGISTRY))["models"]:
        print(f"  {m['version']:6s} {m['stage']:11s} AUC {m['metrics']['auc']}  {m['notes']}")

best = runs.sort_values("auc", ascending=False).iloc[0]
register(build("logreg", C=1.0).fit(train[FEATURES], train["repaid"]), "1.0.0", {"auc": 0.88},
"logistic regression")
promote("1.0.0")
register(build("boosting", max_depth=2, n_estimators=150).fit(train[FEATURES],
train["repaid"]), "1.1.0",
         {"auc": float(runs.loc[runs.kind == "boosting", "auc"].iloc[0])}, "gradient boosting")
promote("1.1.0")
print("after promoting 1.1.0:"); show()

promote("1.0.0")                        # a problem was found: roll back in one step
print("after rollback:"); show()
print("serving:", production()["version"], production()["path"])
```

The serving code only ever asks the registry "which model is in production?". Promotion and rollback become one small, recorded action — not copying files by hand at midnight.

Real tools

- **MLflow** (open source) tracks runs and has a model registry with stages; it runs on your own server.
- Other options: Weights & Biases, DVC, cloud ML platforms.
- Whatever you use: record config, data fingerprint, code version (Git commit), metrics and who approved each promotion.

Key terms

- **Experiment tracking** — recording settings, data and results of every run.
- **Run** — one training attempt.
- **Model registry** — the list of model versions with their stage.
- **Stage** — staging, production or archived.
- **Promotion** — moving a version to production.
- **Rollback** — returning to a previous version.

Remember this: Track every run with its config, data and metrics; keep model versions in a registry so promotion and rollback are single, recorded steps.

Name: _____ Date: _____

Activity

Task 1

Add a `git_commit` field to each run record (use any text for now).

Task 2

Add three more runs and find the fastest model whose AUC is within 0.005 of the best.

Task 3

Add an `approved_by` field to `promote` and refuse promotion when it is empty.

Task 4

Read the MLflow quick-start guide and list which of our functions it replaces.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What should experiment tracking record for every run?
   A. Config, data fingerprint, metrics and time
   B. Only the best score
   C. Nothing
   D. The programmer's mood
2. What does a model registry keep?
   A. Videos
   B. Model versions and which stage each is in
   C. User passwords
   D. Training data
3. Why was the forest with no depth limit worse?
   A. It was too simple
   B. It had no data
   C. Random forests never work
   D. It overfitted the training data
4. When two models have almost the same AUC, which should you usually choose?
   A. The most complex
   B. The slowest
   C. The newest
   D. The simplest, easier to explain and cheaper
5. What is a rollback?
   A. Deleting all models
   B. Returning production to a previous version
   C. Training again
   D. A type of chart
6. Which open-source tool tracks runs and has a model registry?
   A. Paint
   B. Excel
   C. Zoom
   D. MLflow

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.